

## ✓ Clase 24 — Bonus track: cómo funciona RAG por dentro

Diplomado IA Aplicada al Diseño · UDD · 2026 · Prof. Darío Osorio

Este es el **último notebook de Colab del curso**. Es opcional pero recomendado. Muestra en 20 minutos cómo funciona por dentro el patrón que usa NotebookLM (y muchas otras herramientas).

Después de esta clase pasamos 100% a herramientas SaaS.

### Objetivo

Construir un mini-RAG con 3 documentos ficticios y hacerle preguntas.

```
!pip install -q sentence-transformers numpy
print("Listo.")
```

Listo.

## ✓ Paso 1 — Nuestros "documentos"

En NotebookLM cargas PDFs. Acá simulamos con strings.

```
documentos = [
    """3. Usuario / cliente objetivo
    Familias, parejas, grupos de amigos y compañeros de trabajo que valoran el encuentro en torno a la comida (almuerzos, cenas, la

    """Cantidad de registros por ocupación:
    occupation
    Employed          207
    Student            107
    Self-employed      45
    Unemployed         27
    Retired             14
    Name: count, dtype: int64

    Bienestar mental promedio por ocupación:
    occupation
    Student            15.539252
    Employed           20.188406
    Self-employed      20.477778
    Retired            31.578571
    Unemployed         34.274074
    Name: mental_wellness_index_0_100, dtype: float64""",
]
print(f"{len(documentos)} documentos cargados.")
```

2 documentos cargados.

## ✓ Paso 2 — Convertir documentos a embeddings

```
from sentence_transformers import SentenceTransformer
import numpy as np

modelo = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
embeddings = modelo.encode(documentos)
print(f"Shape: {embeddings.shape}")
print("Cada documento es ahora un vector de 384 dimensiones.")
```

```

modules.json: 100%                                349/349 [00:00<00:00, 26.5kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
config_sentence_transformers.json: 100%              116/116 [00:00<00:00, 9.03kB/s]
README.md: 100%                                    10.5k/10.5k [00:00<00:00, 874kB/s]
sentence_bert_config.json: 100%                    53.0/53.0 [00:00<00:00, 4.24kB/s]
config.json: 100%                                  612/612 [00:00<00:00, 19.2kB/s]
model.safetensors: reconstructing file: 100%        90.9MB / 90.9MB, 8.65MB/s
model.safetensors: downloading bytes:              85.0MB, 7.77MB/s
Loading weights: 100%                              103/103 [00:00<00:00, 2226.07it/s]
tokenizer_config.json: 100%                        350/350 [00:00<00:00, 6.39kB/s]
vocab.txt: 100%                                    232k/232k [00:00<00:00, 2.56MB/s]
tokenizer.json: 100%                               466k/466k [00:00<00:00, 11.1MB/s]
special_tokens_map.json: 100%                      112/112 [00:00<00:00, 1.46kB/s]
config.json: 100%                                  190/190 [00:00<00:00, 4.05kB/s]
Shape: (2, 384)
Cada documento es ahora un vector de 384 dimensiones.

```

### ▼ Paso 3 — Función de búsqueda semántica

Le damos una pregunta, ella busca el documento más parecido.

```

def buscar(pregunta, top_k=1):
    p_emb = modelo.encode([pregunta])
    sims = np.dot(embeddings, p_emb.T).flatten()
    sims = sims / (np.linalg.norm(embeddings, axis=1) * np.linalg.norm(p_emb))
    top_idx = np.argsort(sims)[::-1][:top_k]
    return [(documentos[i], float(sims[i])) for i in top_idx]

preguntas = [
    "cuál es el cliente objetivo?",
    "hay algo sobre el bienestar mental?",
    "cuántos estudiantes hay?",
]

for q in preguntas:
    print(f"P: {q}")
    for doc, sim in buscar(q, top_k=1):
        print(f"    Similitud: {sim:.2f}")
        print(f"    Documento: {doc[:150]}...")
    print()

```

```

P: cuál es el cliente objetivo?
Similitud: 0.59
Documento: 3. Usuario / cliente objetivo
Familias, parejas, grupos de amigos y companeros de trabajo que valoran el encuentro en torno a la comida (almuerzos, ce...

P: hay algo sobre el bienestar mental?
Similitud: 0.39
Documento: 3. Usuario / cliente objetivo
Familias, parejas, grupos de amigos y companeros de trabajo que valoran el encuentro en torno a la comida (almuerzos, ce...

P: cuántos estudiantes hay?
Similitud: 0.47
Documento: 3. Usuario / cliente objetivo
Familias, parejas, grupos de amigos y companeros de trabajo que valoran el encuentro en torno a la comida (almuerzos, ce...

```

### Cierre

Esto es exactamente lo que hace NotebookLM (más pulido y con Gemini razonando encima). Ahora que viste el mecanismo, ya sabes por qué:

- Cuando la respuesta "no está en las fuentes", el sistema puede decir "no sé".

- Las citas son literales (vienen del documento recuperado).
- La calidad depende del corte (chunking) y de qué tan buenas son tus fuentes.

**No hace falta que ejecutes esto en tu proyecto** — usa NotebookLM directamente. Este notebook es solo para entender qué pasa por debajo.